EXPT NO: 5	DYNAMIC PROGRAMMING – KNAPSACK PROBLEM, SUBSET SUM PROBLEM, LONGEST COMMON SUBSEQUENCE PROBLEM
DATE:	

AIM

1. To maximize the total value of items included in a knapsack without exceeding its capacity, given weights and values of the items.
2. To determine if a subset of a given set of integers sums up to a specified value.
3. To find the length of the longest subsequence common to two sequences, where the subsequence maintains the order of elements.

PROCEDURE:

1. **KNAPSACK PROBLEM:**

Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

CODE:

PROGRAM 1:

```
def knapsack(weights, values, W):
    n = len(weights)

    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif weights[i-1] <= w:
                dp[i][w] = max(
                    values[i-1] + dp[i-1][w - weights[i-1]],
                    dp[i-1][w]
                )
            else:
                dp[i][w] = dp[i-1][w]
```

```
    return dp[n][W]


# Example usage
weights = [2, 3, 5, 6]
values = [3, 5, 8, 9]
W = 10

print(knapsack(weights, values, W))
```

OUTPUT:

16

PROGRAM 2:

```
def knapsack(weights, values, W):
    n = len(weights)

    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif weights[i-1] <= w:
                dp[i][w] = max(
                    values[i-1] + dp[i-1][w - weights[i-1]],
                    dp[i-1][w]
                )
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][W]


# Example usage
weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
W = 7

print(knapsack(weights, values, W))
```

OUTPUT:

9

2. Subset Sum Problem

Determine if there is a subset of the given set with a sum equal to a given sum.

CODE:

PROGRAM 1:

```
def subset_sum(arr, target_sum):
    n = len(arr)

    dp = [[False for _ in range(target_sum + 1)] for _ in range(n + 1)]

    # Sum = 0 is always possible
    for i in range(n + 1):
        dp[i][0] = True

    for i in range(1, n + 1):
        for s in range(1, target_sum + 1):
            if arr[i-1] <= s:
                dp[i][s] = dp[i-1][s] or dp[i-1][s - arr[i-1]]
            else:
                dp[i][s] = dp[i-1][s]

    return dp[n][target_sum]


# Example usage
arr = [3, 34, 4, 12, 5, 2]
target_sum = 9

print(subset_sum(arr, target_sum))
```

OUTPUT:

True

PROGRAM 2:

```
def subset_sum(arr, target_sum):
    n = len(arr)

    # Create DP table
    dp = [[False for _ in range(target_sum + 1)] for _ in range(n + 1)]

    # Sum = 0 is always possible
    for i in range(n + 1):
        dp[i][0] = True

    # Fill DP table
    for i in range(1, n + 1):
        for s in range(1, target_sum + 1):
            if arr[i-1] <= s:
                dp[i][s] = dp[i-1][s] or dp[i-1][s - arr[i-1]]
            else:
                dp[i][s] = dp[i-1][s]

    return dp[n][target_sum]


# ----------- MAIN PROGRAM -----------

arr = [2, 4, 6, 10]
target_sum = 16

result = subset_sum(arr, target_sum)

print("Array:", arr)
print("Target Sum:", target_sum)

if result:
    print("Subset with given sum exists ")
else:
    print("Subset with given sum does NOT exist ")
```

OUTPUT:

Array: [2, 4, 6, 10]
Target Sum: 16
Subset with given sum exists

3. Longest Common Subsequence Problem

Given two sequences, find the length of the longest subsequence present in both of them.

CODE:

PROGRAM 1:

```
def lcs(X, Y):
    m = len(X)
    n = len(Y)

    dp = [[0 for _ in range(n + 1)] for _ in range(m + 1)]

    for i in range(m + 1):
        for j in range(n + 1):
            if i == 0 or j == 0:
                dp[i][j] = 0
            elif X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]


# Example usage
X = "AGGTAB"
Y = "GXTXAYB"

print(lcs(X, Y))
```

OUTPUT:

4

PROGRAM 2:

```
def lcs(X, Y):
    m = len(X)
    n = len(Y)

    dp = [[0 for _ in range(n + 1)] for _ in range(m + 1)]

    for i in range(m + 1):
```

```
        for j in range(n + 1):
            if i == 0 or j == 0:
                dp[i][j] = 0
            elif X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]


# ----------- MAIN PROGRAM -----------

X = "ABCBDAB"
Y = "BDCABA"

result = lcs(X, Y)
print("Length of LCS:", result)
```

OUTPUT:

4

RESULT:

Thus, The Dynamic Programming Problem Has been Completed.